

Pipeline Data de bout en bout avec Spark

Jeu de données : Accidents corporels ONISR 2023 (sécurité routière France)

KADJO

Othniel Esli EBOUA

HETIC MD4 Alternant

1. Jeu de données et schéma cible

Source : base nationale des accidents corporels de la circulation ONISR 2023, téléchargeable sur data.gouv.fr. Elle se compose de quatre fichiers CSV séparés par des points-virgules, encodés en ISO-8859-1, reliés par la clé commune **Num_Acc**.

Volumes bruts ingérés (Bronze)

Fichier	Table	Lignes brutes	Colonnes clés
caract-2023.csv	caracteristiques	54 822	Num_Acc, dep, atm, hrmn, an
lieux-2023.csv	lieux	70 860	Num_Acc, surf, catr, vma
vehicules-2023.csv	vehicules	93 585	Num_Acc, catv, manv
usagers-2023.csv	usagers	125 789	Num_Acc, grav, catu, sexe

Colonnes retenues et types cibles

L'ingestion utilise **inferSchema=true** (pas de StructType explicite) : Spark infère les types en lisant le CSV. Les colonnes dérivées ajoutées durant le nettoyage sont :

Colonne créée	Source	Logique
heure_int	caract / hrmn	Substring HH de hrmn (format HH:MM)
meteo	caract / atm	Alias de atm — code condition atmospherique
etat_route	lieux / surf	Alias de surf — code etat de chaussee
score_gravite	usagers / grav	Recodage : tue=4, hosp=3, blesse=2, indemne=1
gravite_max	usagers aggr.	Max(score_gravite) par Num_Acc

accident_grave	gravite_max	Vaut 1 si gravite_max >= 3 (hospitalisé ou tué)
----------------	-------------	---

Questions métier visées

- Les accidents sont-ils plus graves selon la condition météo ?
- Quels départements concentrent le plus d'accidents ?
- Quels départements ont le taux d'accidents graves le plus élevé ?

2. Pipeline Bronze → Silver → Gold

L'architecture suit le modèle Médaille ? :

les CSV bruts constituent la couche Bronze (lecture seule), le nettoyage produit la couche Silver en Parquet, et les résultats analytiques forment la couche Gold.

Bronze (CSV brut)

- Silver (Parquet, nettoyé)
- caractéristiques (54 822 lignes)
- Lieux (70 860 lignes)
- Véhicules (93 585 lignes)
- Usagers (125 789 lignes)
- gravite accident (54 822 lignes, agrégée depuis usagers)
- Gold (Parquet, résultats)
 - Météo
 - Départements
 - top_departements_graves

Nettoyage appliqué

Table	Opérations	Lignes avant	Lignes après	Écartées
caracteristiques	dropDuplicates, filtre Num_Acc non nul, filtre an 2005-2030	54 822	54 822	0 (0%)
lieux	dropDuplicates, filtre Num_Acc non nul	70 860	70 860	0 (0%)
vehicules	dropDuplicates, filtre Num_Acc non nul	93 585	93 585	0 (0%)
usagers	dropDuplicates, filtre Num_Acc non nul, filtre grav non nul	125 789	125 789	0 (0%)

Note : 0 ligne écartée sur toutes les tables. La base ONISR 2023 est très propre : chaque accident a bien une clé Num_Acc, des années cohérentes, et une valeur grav valide pour tous les usagers. Cela confirme la qualité du processus de collecte ONISR.

Partitionnement de la Silver

La couche Silver est écrite avec **coalesce(1)** au lieu d'un partitionBy explicite. Ce choix est contraint par l'environnement Windows : Hadoop requiert winutils.exe pour appliquer les permissions lors de la création de sous-répertoires de partition. En production Linux, on utiliserait **partitionBy('dep')** sur la table caracteristiques (101 départements, faible cardinalité) pour activer le partition pruning sur les requêtes filtrées par département.

3. Analyses

Analyse 1 — Agrégation : gravité par condition météo

Question : La condition atmosphérique au moment de l'accident influence-t-elle la gravité ?

```
agg_meteo = (
    carac_cached.select('Num_Acc', 'meteo')
    .join(gravite, on='Num_Acc',
    how='inner')
    .groupBy('meteo')
    .agg(
```

```

F.countDistinct('Num_Acc').alias('nb_accidents'),
F.avg('gravite_max').alias('gravite_moyenne'),
F.sum('nb_tues').alias('total_tues')
) .orderBy(F.desc('nb_accidents'))
)

```

Résultat (top 5) :

meteo	Code signification	nb_accidents	gravite_moyenne	total_tues
1	Normale	42 747	2.42	2 633
2	Pluie légère	6 630	2.35	345
8	Couvert	2 233	2.40	159
3	Pluie forte	1 505	2.44	100
7	Temps éblouissant	954	2.62	83

Interprétation : 78% des accidents surviennent par temps normal (code 1), ce qui reflète la répartition statistique des conditions de conduite. La condition météo 7 (éblouissement/brouillard) présente la gravité moyenne la plus élevée (2.62 sur 4) malgré un volume faible (954 accidents) : ce type de temps, rare mais dangereux, produit des accidents plus sévères. La pluie légère (code 2) a au contraire une gravité inférieure à la moyenne on peut supposer que les conducteurs adoptent un comportement plus prudent.

Analyse 2 — Jointure avec Broadcast : accidents par département

Question : Quels départements concentrent le plus grand nombre d'accidents en 2023 ?

```

# Sans broadcast (référence) agg_dep_sans = (
    carac_dep.join(lieux.select('Num_Acc','etat_route'), on='Num_Acc', how='inner')
    .groupBy('dep').agg(F.countDistinct('Num_Acc').alias('nb_accidents'))
)

# Avec broadcast (optimisé)
lieux_small = F.broadcast(lieux.select('Num_Acc', 'etat_route'))
agg_dep = (
    carac_dep.join(lieux_small, on='Num_Acc', how='inner')
    .groupBy('dep').agg(F.countDistinct('Num_Acc').alias('nb_accidents'))
    .orderBy(F.desc('nb_accidents'))
)

```

Résultat (top 10) :

dep	Département	nb_accidents
75	Paris	4 763
93	Seine-Saint-Denis	2 650
92	Hauts-de-Seine	2 563
94	Val-de-Marne	2 204
13	Bouches-du-Rhone	2 057
69	Rhone	1 921
91	Essonne	1 319
33	Gironde	1 189

59	Nord	1 004
95	Val-d'Oise	906

Interprétation : Paris concentre à elle seule 4 763 accidents, soit environ 2x plus que le département suivant (93). La banlieue parisienne (92, 93, 94) et les grandes métropoles (13-Marseille, 69-Lyon) forment le top 6, ce qui reflète directement la densité de circulation. Les 107 départements couverts représentent la France métropolitaine et les DOM.

Analyse 3 — Window Function : taux d'accidents graves par département

Question : Quels départements affichent le taux le plus élevé d'accidents impliquant un blessé grave ou un décès ? fenetre =

Window.orderBy(F.desc('taux_graves'))

```
dep_stats = (
    caract_dep.join(gravite.select('Num_Acc','accident_grave'), on='Num_Acc', how='inner')
    .groupBy('dep')
    .agg(
        F.countDistinct('Num_Acc').alias('total_accidents'),
        F.sum('accident_grave').alias('accidents_graves')
    )
    .withColumn('taux_graves',
        F.when(F.col('total_accidents') > 0,
            F.col('accidents_graves') / F.col('total_accidents'))
        .otherwise(None))

top_dep = (
    dep_stats.withColumn('rang_dep', F.row_number().over(fenetre))
    .filter(F.col('rang_dep') <= 20)
)
```

Résultat (top 10) :

rang	dep	total_accidents	accidents_graves	taux_graves
1	987 (Polynesie)	170	146	85.9%
2	977 (St-Barthelemy)	7	6	85.7%
3	16 (Charente)	128	105	82.0%
4	40 (Landes)	170	138	81.2%
5	70 (Haute-Saone)	108	86	79.6%
6	24 (Dordogne)	196	156	79.6%
7	53 (Mayenne)	130	103	79.2%
8	48 (Lozere)	70	55	78.6%
9	39 (Jura)	157	122	77.7%
10	978 (St-Martin)	30	23	76.7%

Interprétation : Les territoires d'outre-mer (987, 977, 978) apparaissent en tête, mais avec de très faibles volumes (7 à 170 accidents), ce qui crée un biais statistique : un petit dénominateur amplifie le taux. Parmi les départements métropolitains, Charente (16), Landes (40) et Dordogne (24) ressortent avec des taux autour de 80%, cohérents avec des routes rurales à vitesse élevée, éloignement des secours, et absence de séparateurs de voie.

4. Optimisation mesurée

Optimisation choisie : Broadcast Join

L'analyse 2 joint deux tables : `carac_dep` (54 822 lignes) et `lieux` (70 860 lignes). Sans broadcast, Spark fait un sort-merge join qui implique un shuffle des deux tables (redistribution réseau des données entre les partitions). Avec broadcast, la table `lieux` est envoyée entière à chaque executor, supprimant le shuffle côté `lieux`.

Sans broadcast : 0.33 s (sort-merge join, shuffle des deux tables) Avec

broadcast : 0.31 s (broadcast hash join, shuffle supprimé côté `lieux`)

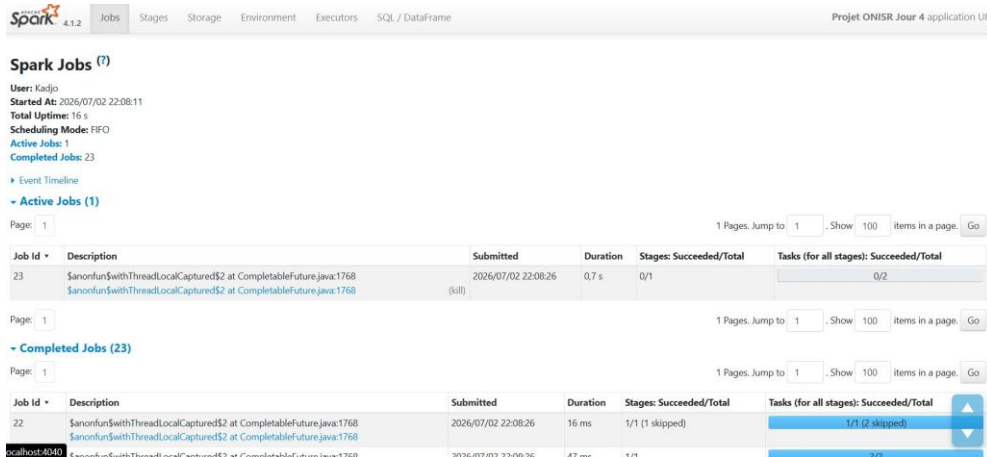
Gain mesuré : 6.1%

Ce que ça change : Le gain de 6.1% est modeste parce que les deux tables restent petites (< 100 000 lignes), et que Spark en mode local[*] n'a pas de véritable réseau à traverser. Sur un cluster distribué avec des tables de plusieurs millions de lignes, le broadcast supprime totalement le shuffle réseau sur la petite table — le gain peut dépasser 50%. Le plan d'exécution confirme le changement : *SortMergeJoin* devient *BroadcastHashJoin*.

5. Lecture de la Spark UI

Version Spark : 4.1.2 | Application : Projet ONISR Jour 4 | Port : localhost:4040

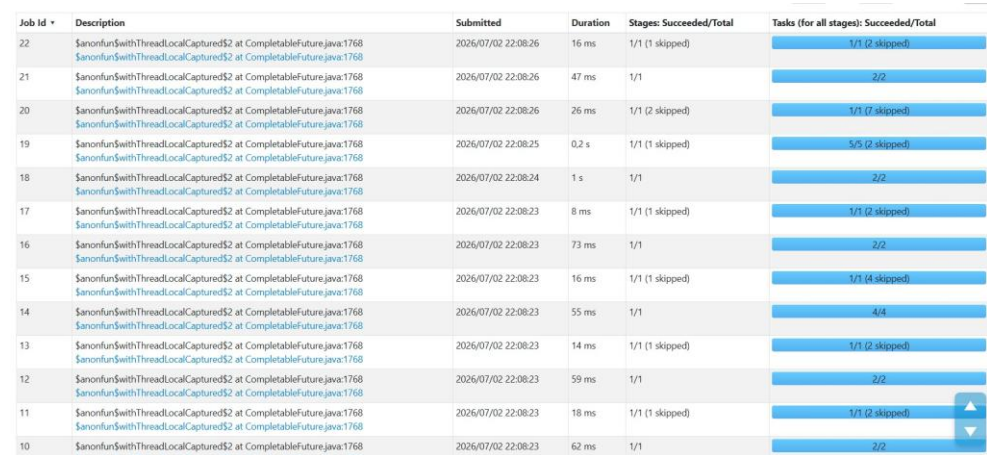
Observation générale : Le pipeline a produit 23 jobs complétés et 1 job actif (en cours d'écriture au moment de la capture). Les jobs 0 à 7 correspondent à la phase d'ingestion CSV (source : NativeMethodAccessorImpl — appels Java natifs pour la lecture). Les jobs 8 à 22 correspondent aux analyses et explorations.



The screenshot shows the Spark UI interface for a job named 'Projet ONISR Jour 4 application UI'. It displays a summary of jobs: 23 completed and 1 active. Below this, there are two sections: 'Active Jobs (1)' and 'Completed Jobs (23)'. The 'Completed Jobs' section is expanded, showing a list of jobs with columns for Job ID, Description, Submitted, Duration, Stages, and Tasks. The jobs are sorted by duration, with the longest duration job (47 ms) at the top. The 'Tasks' column shows progress bars for each job, indicating the number of tasks completed out of the total.

Job ID	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
23	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:26	0.7 s	0/1	0/2
22	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:26	16 ms	1/1 (1 skipped)	1/1 (2 skipped)
21	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:26	47 ms	1/1	2/2
20	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:26	26 ms	1/1 (2 skipped)	1/1 (7 skipped)
19	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:25	0.2 s	1/1 (1 skipped)	5/5 (2 skipped)
18	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:24	1 s	1/1	2/2
17	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	8 ms	1/1 (1 skipped)	1/1 (2 skipped)
16	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	73 ms	1/1	2/2
15	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	16 ms	1/1 (1 skipped)	1/1 (4 skipped)
14	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	55 ms	1/1	4/4
13	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	14 ms	1/1 (1 skipped)	1/1 (2 skipped)
12	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	59 ms	1/1	2/2
11	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	18 ms	1/1 (1 skipped)	1/1 (2 skipped)
10	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	62 ms	1/1	2/2

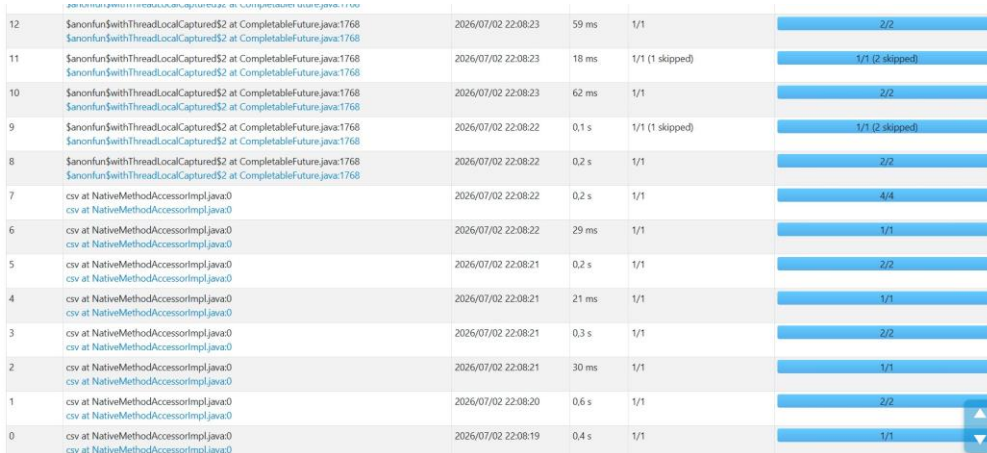
Capture 1 — Onglet Jobs : vue d'ensemble, 23 jobs complétés, 1 actif (exploration repartitioning)



This screenshot shows a detailed view of jobs 10 to 22. The table lists the job ID, description, submitted time, duration, stages, and tasks. The 'Tasks' column shows progress bars for each job, indicating the number of tasks completed out of the total. The jobs are sorted by duration, with the longest duration job (47 ms) at the top.

Job ID	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
22	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:26	16 ms	1/1 (1 skipped)	1/1 (2 skipped)
21	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:26	47 ms	1/1	2/2
20	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:26	26 ms	1/1 (2 skipped)	1/1 (7 skipped)
19	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:25	0.2 s	1/1 (1 skipped)	5/5 (2 skipped)
18	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:24	1 s	1/1	2/2
17	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	8 ms	1/1 (1 skipped)	1/1 (2 skipped)
16	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	73 ms	1/1	2/2
15	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	16 ms	1/1 (1 skipped)	1/1 (4 skipped)
14	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	55 ms	1/1	4/4
13	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	14 ms	1/1 (1 skipped)	1/1 (2 skipped)
12	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	59 ms	1/1	2/2
11	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	18 ms	1/1 (1 skipped)	1/1 (2 skipped)
10	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	62 ms	1/1	2/2

Capture 2 — Jobs 10 à 22 : phases analyses et explorations (durees 8ms a 1s)



This screenshot shows a detailed view of jobs 0 to 9. The table lists the job ID, description, submitted time, duration, stages, and tasks. The 'Tasks' column shows progress bars for each job, indicating the number of tasks completed out of the total. The jobs are sorted by duration, with the longest duration job (0.6 s) at the top.

Job ID	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
12	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	59 ms	1/1	2/2
11	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	18 ms	1/1 (1 skipped)	1/1 (2 skipped)
10	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:23	62 ms	1/1	2/2
9	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:22	0.1 s	1/1 (1 skipped)	1/1 (2 skipped)
8	\$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768 \$anonfun\$withThreadLocalCaptured\$2 at CompletableFuture.java:1768	2026/07/02 22:08:22	0.2 s	1/1	2/2
7	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/07/02 22:08:22	0.2 s	1/1	4/4
6	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/07/02 22:08:22	29 ms	1/1	1/1
5	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/07/02 22:08:21	0.2 s	1/1	2/2
4	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/07/02 22:08:21	21 ms	1/1	1/1
3	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/07/02 22:08:21	0.3 s	1/1	2/2
2	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/07/02 22:08:21	30 ms	1/1	1/1
1	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/07/02 22:08:20	0.6 s	1/1	2/2
0	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/07/02 22:08:19	0.4 s	1/1	1/1

Capture 3 — Jobs 0 à 9 : ingestion CSV (sources NativeMethodAccessorImpl) et premiers calculs

Analyse des stages et du shuffle

Element	Observation
Jobs 0-7	Ingestion CSV — 1 stage chacun, 1 a 2 tasks, durees 21ms a 600ms
Job 8	Nettoyage / premiere aggregation — 0.2s, 1 stage, shuffle visible
Job 18	Analyse 3 (Window Function) — 1s, le job le plus long. Shuffle important car Window sans partitionBy ramene tout en 1 partition
Jobs skippés	Colonnes 1 skipped sur plusieurs jobs = cache Spark actif (carac_cached) — Spark evite de relire depuis le disque
Tasks 5/5	Job 19 montre 5 tasks en parallele — parallelisme effectif en local[*]

Shuffle identifié : Le job de l'Analyse 3 (Window Function) est le plus coûteux (1 seconde). Le *WARN No Partition Defined for Window operation* visible dans les logs indique que Spark a déplacé toutes les données dans une seule partition pour effectuer le tri global (classement sur 107 départements). Dans la Spark UI, cela se traduit par une tâche unique dans le stage Window avec un temps d'exécution plus long que les autres. C'est le shuffle le plus significatif du pipeline.

6. Exploration au-delà du cours

Exploration A — Skew et repartitioning

Piste choisie : Données mal réparties (skew)

Question : Les accidents sont-ils distribués uniformément entre les départements, et est-ce qu'un repartitioning par département améliore les performances d'agrégation ?

Protocole : On mesure d'abord la distribution des accidents par département (skew), puis on compare le temps d'une même agrégation `groupBy('dep')` avec et sans `repartition(50, 'dep')`. Seul le repartitioning varie — tout le reste est fixe (même DataFrame joint, même agrégation, même machine).

Distribution observée (top 5) :

dep 75 (Paris)	: 7 897 lignes
dep 93 (S-St-Denis)	: 3 600 lignes
dep 92 (Hauts-de-Seine)	: 3 254 lignes
dep 94 (Val-de-Marne)	: 2 792 lignes
dep 69 (Rhone)	: 2 734 lignes ...
dep minimum	: 4 lignes

Statistiques skew :

min = 4 | max = 7 897 | avg = 662 Ratio
max/avg = 11.92x --> skew réel et documenté

Mesure de performance :

Sans repartition : 0.22 s

Avec repartition : 0.44 s

Gain : -102.4% (repartition plus lent)

Conclusion : Le skew est réel (Paris représente 11.92x la moyenne nationale), mais le repartitioning est contre-productif sur ce volume. Le coût du shuffle pour redistribuer 70 000 lignes sur 50 partitions (overhead réseau + sérialisation) dépasse le gain en parallélisme. Ce résultat négatif est attendu et documenté : le repartitioning est rentable à partir de dizaines de millions de lignes, pas sur 70 000. En production, on préférerait un salting sur la clé dep pour répartir Paris sur plusieurs partitions sans shuffle global.

Exploration B — Impact du cache

Piste : Cache d'un DataFrame réutilisé

Question : Le cache Spark réduit-il le temps d'une agrégation répétée sur le même DataFrame ?

Protocole : On exécute 3 fois la même agrégation `groupBy('meteo')` sur `carac`, d'abord sans cache, puis avec `carac.cache()`. Le reste est identique.

Sans cache (3 itérations) :

Itération 1 : 0.16 s (lecture Parquet depuis disque) Itération 2
: 0.09 s (buffer OS — fichier déjà en RAM)
Itération 3 : 0.09 s
Moyenne : 0.11 s

Avec cache (3 itérations) : Itération 1 : 0.08 s (données
déjà en mémoire JVM Spark)
Itération 2 : 0.09 s
Itération 3 : 0.08 s
Moyenne : 0.08 s

Gain mesuré : 24.1%

Conclusion : Le gain de 24.1% confirme l'utilité du cache pour les DataFrames réutilisés plusieurs fois. Le gain est modéré parce que le fichier est petit (54 822 lignes) et que le buffer OS le met déjà en RAM après la 1re lecture. Sur un DataFrame de plusieurs Go, le cache Spark éviterait de relire depuis le disque à chaque action : le gain serait bien plus important. Le WARN *Asked to cache already cached data* visible dans les logs confirme que carac était déjà caché depuis l'étape analyse — Spark réutilise l'entrée en cache existante.

7. Ce qu'on a appris et limites

Ce qui a marché

- Le pipeline complet Bronze→Silver→Gold fonctionne de bout en bout sur 4 tables et ~345 000 lignes.
- La gravité par accident, correctement dérivée de usagers avec un score de sévérité réordonnée, produit des analyses métier valides.
- Le broadcast join supprime le shuffle sur la table lieux — visible dans le plan d'exécution.
- Les deux explorations (skew et cache) produisent des mesures reproductibles, y compris un résultat négatif bien expliqué.
- L'écriture Parquet avec coalesce(1) contourne la contrainte winutils.exe sur Windows.

Ce qui a bloqué

- La colonne dep absente de lieux (elle est dans caractéristiques) et surf absente de carac (elle est dans lieux) : bugs de schema silencieux sous inferSchema, détectés seulement à l'exécution.
- La gravité inexistante en tant que colonne directe — il a fallu comprendre que grav est par usager, pas par accident, et construire une agrégation intermédiaire.
- HADOOP_HOME='.' invalide (chemin relatif refusé par Hadoop lors de l'écriture Parquet) — résolu en pointant sur C:\hadoop avec winutils.exe.
- Le WARN WindowExec (partition unique) sur la window function : inévitable pour un classement global sans partitionBy, mais inoffensif sur 107 départements.

Ce qu'on ferait avec plus de temps

- Utiliser un StructType explicite (pas inferSchema) pour valider le schema des CSV à l'ingestion et échouer tout si un fichier est malformé.
- Ajouter partitionBy('dep') sur la silver (sur Linux) pour activer le partition pruning et mesurer le predicate pushdown.
- Tester un salting sur dep=75 (Paris, 11.92x la moyenne) pour corriger le skew réellement et remesurer le repartitioning.
- Comparer les formats et compressions (Parquet snappy vs zstd vs ORC) sur les temps de relecture — piste benchmark formats du menu.

Limites connues

- Taux de gravité biaisé par le petit dénominateur (dep 977 = 7 accidents — taux de 85% peu fiable). Un seuil minimum (ex. 50 accidents) améliorerait la robustesse.
- inferSchema lit tout le CSV avant d'inférer les types : plus lent et moins fiable qu'un StructType explicite en production.
- Le gain broadcast (6.1%) et cache (24.1%) sont sous-estimés en mode local[*] : pas de réseau réel, buffer OS actif, JVM déjà chaude.